

1. Pythonic Thinking

The idioms of a programming language are defined by its users. Over the years, the Python community has come to use the adjective *Pythonic* to describe code that follows a particular style. The Pythonic style isn't regimented or enforced by the compiler. It has emerged over time through experience using the language and working with others. Python programmers prefer to be explicit, to choose simple over complex, and to maximize readability (type `import this`).

Programmers familiar with other languages may try to write Python as if it's C++, Java, or whatever they know best. New programmers may still be getting comfortable with the vast range of concepts expressible in Python. It's important for everyone to know the best—the *Pythonic*—way to do the most common things in Python. These patterns will affect every program you write.

Item 1: Know Which Version of Python You're Using

Throughout this book, the majority of example code is in the syntax of Python 3.4 (released March 17, 2014). This book also provides some examples in the syntax of Python 2.7 (released July 3, 2010) to highlight important differences. Most of my advice applies to all of the popular Python runtimes: CPython, Jython, IronPython, PyPy, etc.

Many computers come with multiple versions of the standard CPython runtime preinstalled. However, the default meaning of `python` on the command-line may not be clear. `python` is usually an alias for `python2.7`, but it can sometimes be an alias for older versions like `python2.6` or `python2.5`. To find out exactly which version of Python you're using, you can use the `--version` flag.

```
$ python --version
Python 2.7.8
```

Python 3 is usually available under the name `python3`.

```
$ python3 --version
Python 3.4.2
```

You can also figure out the version of Python you're using at runtime by inspecting values in the `SYS` built-in module.

[Click here to view code image](#)

```
import sys
print(sys.version_info)
print(sys.version)

>>>
sys.version_info(major=3, minor=4, micro=2, releaselevel='final', serial=0)
3.4.2 (default, Oct 19 2014, 17:52:17)
[GCC 4.2.1 Compatible Apple LLVM 6.0 (clang-600.0.51)]
```

Python 2 and Python 3 are both actively maintained by the Python community. Development on Python 2 is frozen beyond bug fixes, security improvements, and backports to ease the transition from Python 2 to Python 3. Helpful tools like the `2to3`

and `six` exist to make it easier to adopt Python 3 going forward.

Python 3 is constantly getting new features and improvements that will never be added to Python 2. As of the writing of this book, the majority of Python's most common open source libraries are compatible with Python 3. I strongly encourage you to use Python 3 for your next Python project.

Things to Remember

- ◆ There are two major versions of Python still in active use: Python 2 and Python 3.
- ◆ There are multiple popular runtimes for Python: CPython, Jython, IronPython, PyPy, etc.
- ◆ Be sure that the command-line for running Python on your system is the version you expect it to be.
- ◆ Prefer Python 3 for your next project because that is the primary focus of the Python community.

Item 2: Follow the PEP 8 Style Guide

Python Enhancement Proposal #8, otherwise known as PEP 8, is the style guide for how to format Python code. You are welcome to write Python code however you want, as long as it has valid syntax. However, using a consistent style makes your code more approachable and easier to read. Sharing a common style with other Python programmers in the larger community facilitates collaboration on projects. But even if you are the only one who will ever read your code, following the style guide will make it easier to change things later.

PEP 8 has a wealth of details about how to write clear Python code. It continues to be updated as the Python language evolves. It's worth reading the whole guide online (<http://www.python.org/dev/peps/pep-0008/>). Here are a few rules you should be sure to follow:

Whitespace: In Python, whitespace is syntactically significant. Python programmers are especially sensitive to the effects of whitespace on code clarity.

- Use spaces instead of tabs for indentation.
- Use four spaces for each level of syntactically significant indenting.
- Lines should be 79 characters in length or less.
- Continuations of long expressions onto additional lines should be indented by four extra spaces from their normal indentation level.
- In a file, functions and classes should be separated by two blank lines.
- In a class, methods should be separated by one blank line.
- Don't put spaces around list indexes, function calls, or keyword argument assignments.
- Put one—and only one—space before and after variable assignments.

Naming: PEP 8 suggests unique styles of naming for different parts in the language. This makes it easy to distinguish which type corresponds to each name when reading code.

- Functions, variables, and attributes should be in `lowercase_underscore` format.
- Protected instance attributes should be in `_leading_underscore` format.
- Private instance attributes should be in `__double_leading_underscore` format.
- Classes and exceptions should be in `CapitalizedWord` format.
- Module-level constants should be in `ALL_CAPS` format.
- Instance methods in classes should use `self` as the name of the first parameter (which refers to the object).
- Class methods should use `cls` as the name of the first parameter (which refers to the class).

Expressions and Statements: *The Zen of Python* states: “There should be one—and preferably only one—obvious way to do it.” PEP 8 attempts to codify this style in its guidance for expressions and statements.

- Use inline negation (`if a is not b`) instead of negation of positive expressions (`if not a is b`).
- Don’t check for empty values (like `[]` or `''`) by checking the length (`if len(somelist) == 0`). Use `if not somelist` and assume empty values implicitly evaluate to `False`.
- The same thing goes for non-empty values (like `[1]` or `'hi'`). The statement `if somelist` is implicitly `True` for non-empty values.
- Avoid single-line `if` statements, `for` and `while` loops, and `except` compound statements. Spread these over multiple lines for clarity.
- Always put `import` statements at the top of a file.
- Always use absolute names for modules when importing them, not names relative to the current module’s own path. For example, to import the `foo` module from the `bar` package, you should do `from bar import foo`, not just `import foo`.
- If you must do relative imports, use the explicit syntax `from . import foo`.
- Imports should be in sections in the following order: standard library modules, third-party modules, your own modules. Each subsection should have imports in alphabetical order.

Note

The Pylint tool (<http://www.pylint.org/>) is a popular static analyzer for Python source code. Pylint provides automated enforcement of the PEP 8 style guide and detects many other types of common errors in Python programs.

Things to Remember

- ◆ Always follow the PEP 8 style guide when writing Python code.
- ◆ Sharing a common style with the larger Python community facilitates collaboration with others.
- ◆ Using a consistent style makes it easier to modify your own code later.

Item 3: Know the Differences Between `bytes`, `str`, and `unicode`

In Python 3, there are two types that represent sequences of characters: `bytes` and `str`. Instances of `bytes` contain raw 8-bit values. Instances of `str` contain Unicode characters.

In Python 2, there are two types that represent sequences of characters: `str` and `unicode`. In contrast to Python 3, instances of `str` contain raw 8-bit values. Instances of `unicode` contain Unicode characters.

There are many ways to represent Unicode characters as binary data (raw 8-bit values). The most common encoding is *UTF-8*. Importantly, `str` instances in Python 3 and `unicode` instances in Python 2 do not have an associated binary encoding. To convert Unicode characters to binary data, you must use the `encode` method. To convert binary data to Unicode characters, you must use the `decode` method.

When you're writing Python programs, it's important to do encoding and decoding of Unicode at the furthest boundary of your interfaces. The core of your program should use Unicode character types (`str` in Python 3, `unicode` in Python 2) and should not assume anything about character encodings. This approach allows you to be very accepting of alternative text encodings (such as *Latin-1*, *Shift JIS*, and *Big5*) while being strict about your output text encoding (ideally, *UTF-8*).

The split between character types leads to two common situations in Python code:

- You want to operate on raw 8-bit values that are *UTF-8*-encoded characters (or some other encoding).
- You want to operate on Unicode characters that have no specific encoding.

You'll often need two helper functions to convert between these two cases and to ensure that the type of input values matches your code's expectations.

In Python 3, you'll need one method that takes a `str` or `bytes` and always returns a `str`.

[Click here to view code image](#)

```
def to_str(bytes_or_str):
    if isinstance(bytes_or_str, bytes):
        value = bytes_or_str.decode('utf-8')
    else:
        value = bytes_or_str
    return value # Instance of str
```

You'll need another method that takes a `str` or `bytes` and always returns a `bytes`.

[Click here to view code image](#)

```
def to_bytes(bytes_or_str):
    if isinstance(bytes_or_str, str):
        value = bytes_or_str.encode('utf-8')
    else:
        value = bytes_or_str
    return value # Instance of bytes
```

In Python 2, you'll need one method that takes a `str` or `unicode` and always returns a `unicode`.

[Click here to view code image](#)

```
# Python 2
def to_unicode(unicode_or_str):
    if isinstance(unicode_or_str, str):
        value = unicode_or_str.decode('utf-8')
    else:
        value = unicode_or_str
    return value # Instance of unicode
```

You'll need another method that takes `str` or `unicode` and always returns a `str`.

[Click here to view code image](#)

```
# Python 2
def to_str(unicode_or_str):
    if isinstance(unicode_or_str, unicode):
        value = unicode_or_str.encode('utf-8')
    else:
        value = unicode_or_str
    return value # Instance of str
```

There are two big gotchas when dealing with raw 8-bit values and Unicode characters in Python.

The first issue is that in Python 2, `unicode` and `str` instances seem to be the same type when a `str` only contains 7-bit ASCII characters.

- You can combine such a `str` and `unicode` together using the `+` operator.
- You can compare such `str` and `unicode` instances using equality and inequality operators.
- You can use `unicode` instances for format strings like `'%s'`.

All of this behavior means that you can often pass a `str` or `unicode` instance to a function expecting one or the other and things will just work (as long as you're only dealing with 7-bit ASCII). In Python 3, `bytes` and `str` instances are never equivalent—not even the empty string—so you must be more deliberate about the types of character

sequences that you're passing around.

The second issue is that in Python 3, operations involving file handles (returned by the `open` built-in function) default to UTF-8 encoding. In Python 2, file operations default to binary encoding. This causes surprising failures, especially for programmers accustomed to Python 2.

For example, say you want to write some random binary data to a file. In Python 2, this works. In Python 3, this breaks.

[Click here to view code image](#)

```
with open('/tmp/random.bin', 'w') as f:
    f.write(os.urandom(10))
```

```
>>>
```

```
TypeError: must be str, not bytes
```

The cause of this exception is the new `encoding` argument for `open` that was added in Python 3. This parameter defaults to `'utf-8'`. That makes `read` and `write` operations on file handles expect `str` instances containing Unicode characters instead of `bytes` instances containing binary data.

To make this work properly, you must indicate that the data is being opened in write binary mode (`'wb'`) instead of write character mode (`'w'`). Here, I use `open` in a way that works correctly in Python 2 and Python 3:

[Click here to view code image](#)

```
with open('/tmp/random.bin', 'wb') as f:
    f.write(os.urandom(10))
```

This problem also exists for reading data from files. The solution is the same: Indicate binary mode by using `'rb'` instead of `'r'` when opening a file.

Things to Remember

- ◆ In Python 3, `bytes` contains sequences of 8-bit values, `str` contains sequences of Unicode characters. `bytes` and `str` instances can't be used together with operators (like `>` or `+`).
- ◆ In Python 2, `str` contains sequences of 8-bit values, `unicode` contains sequences of Unicode characters. `str` and `unicode` *can* be used together with operators if the `str` only contains 7-bit ASCII characters.
- ◆ Use helper functions to ensure that the inputs you operate on are the type of character sequence you expect (8-bit values, UTF-8 encoded characters, Unicode characters, etc.).
- ◆ If you want to read or write binary data to/from a file, always open the file using a binary mode (like `'rb'` or `'wb'`).

Item 4: Write Helper Functions Instead of Complex Expressions

Python's pithy syntax makes it easy to write single-line expressions that implement a lot of logic. For example, say you want to decode the query string from a URL. Here, each query string parameter represents an integer value:

[Click here to view code image](#)

```
from urllib.parse import parse_qs
my_values = parse_qs('red=5&blue=0&green=',
                    keep_blank_values=True)
print(repr(my_values))

>>>
{'red': ['5'], 'green': [], 'blue': ['0']}
```

Some query string parameters may have multiple values, some may have single values, some may be present but have blank values, and some may be missing entirely. Using the `get` method on the result dictionary will return different values in each circumstance.

[Click here to view code image](#)

```
print('Red:      ', my_values.get('red'))
print('Green:     ', my_values.get('green'))
print('Opacity:   ', my_values.get('opacity'))

>>>
Red:      ['5']
Green:    []
Opacity:  None
```

It'd be nice if a default value of `0` was assigned when a parameter isn't supplied or is blank. You might choose to do this with Boolean expressions because it feels like this logic doesn't merit a whole `if` statement or helper function quite yet.

Python's syntax makes this choice all too easy. The trick here is that the empty string, the empty list, and zero all evaluate to `False` implicitly. Thus, the expressions below will evaluate to the subexpression after the `or` operator when the first subexpression is `False`.

[Click here to view code image](#)

```
# For query string 'red=5&blue=0&green='
red = my_values.get('red', [''])[0] or 0
green = my_values.get('green', [''])[0] or 0
opacity = my_values.get('opacity', [''])[0] or 0
print('Red:      %r' % red)
print('Green:     %r' % green)
print('Opacity:   %r' % opacity)

>>>
Red:      '5'
Green:    0
Opacity:  0
```

The `red` case works because the key is present in the `my_values` dictionary. The value is a list with one member: the string `'5'`. This string implicitly evaluates to `True`, so

`red` is assigned to the first part of the `or` expression.

The `green` case works because the value in the `my_values` dictionary is a list with one member: an empty string. The empty string implicitly evaluates to `False`, causing the `or` expression to evaluate to `0`.

The `opacity` case works because the value in the `my_values` dictionary is missing altogether. The behavior of the `get` method is to return its second argument if the key doesn't exist in the dictionary. The default value in this case is a list with one member, an empty string. When `opacity` isn't found in the dictionary, this code does exactly the same thing as the `green` case.

However, this expression is difficult to read and it still doesn't do everything you need. You'd also want to ensure that all the parameter values are integers so you can use them in mathematical expressions. To do that, you'd wrap each expression with the `int` built-in function to parse the string as an integer.

[Click here to view code image](#)

```
red = int(my_values.get('red', ['']))[0] or 0
```

This is now extremely hard to read. There's so much visual noise. The code isn't approachable. A new reader of the code would have to spend too much time picking apart the expression to figure out what it actually does. Even though it's nice to keep things short, it's not worth trying to fit this all on one line.

Python 2.5 added `if/else` conditional—or ternary—expressions to make cases like this clearer while keeping the code short.

[Click here to view code image](#)

```
red = my_values.get('red', [''])
red = int(red[0]) if red[0] else 0
```

This is better. For less complicated situations, `if/else` conditional expressions can make things very clear. But the example above is still not as clear as the alternative of a full `if/else` statement over multiple lines. Seeing all of the logic spread out like this makes the dense version seem even more complex.

[Click here to view code image](#)

```
green = my_values.get('green', [''])
if green[0]:
    green = int(green[0])
else:
    green = 0
```

Writing a helper function is the way to go, especially if you need to use this logic repeatedly.

[Click here to view code image](#)

```
def get_first_int(values, key, default=0):
    found = values.get(key, [''])
    if found[0]:
        found = int(found[0])
    else:
        found = default
    return found
```


The calling code is much clearer than the complex expression using `or` and the two-line version using the `if/else` expression.

[Click here to view code image](#)

```
green = get_first_int(my_values, 'green')
```

As soon as your expressions get complicated, it's time to consider splitting them into smaller pieces and moving logic into helper functions. What you gain in readability always outweighs what brevity may have afforded you. Don't let Python's pithy syntax for complex expressions get you into a mess like this.

Things to Remember

- ◆ Python's syntax makes it all too easy to write single-line expressions that are overly complicated and difficult to read.
- ◆ Move complex expressions into helper functions, especially if you need to use the same logic repeatedly.
- ◆ The `if/else` expression provides a more readable alternative to using Boolean operators like `or` and `and` in expressions.

Item 5: Know How to Slice Sequences

Python includes syntax for slicing sequences into pieces. Slicing lets you access a subset of a sequence's items with minimal effort. The simplest uses for slicing are the built-in types `list`, `str`, and `bytes`. Slicing can be extended to any Python class that implements the `__getitem__` and `__setitem__` special methods (see [Item 28: “Inherit from `collections.abc` for Custom Container Types](#)”).

The basic form of the slicing syntax is `somelist[start:end]`, where `start` is inclusive and `end` is exclusive.

[Click here to view code image](#)

```
a = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
print('First four:', a[:4])
print('Last four: ', a[-4:])
print('Middle two:', a[3:-3])

>>>
First four: ['a', 'b', 'c', 'd']
Last four:  ['e', 'f', 'g', 'h']
Middle two: ['d', 'e']
```

When slicing from the start of a list, you should leave out the zero index to reduce visual noise.

```
assert a[:5] == a[0:5]
```

When slicing to the end of a list, you should leave out the final index because it's redundant.

```
assert a[5:] == a[5:len(a)]
```

Using negative numbers for slicing is helpful for doing offsets relative to the end of a list.

All of these forms of slicing would be clear to a new reader of your code. There are no surprises, and I encourage you to use these variations.

[Click here to view code image](#)

```
a[:]      # ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
a[:5]     # ['a', 'b', 'c', 'd', 'e']
a[:-1]    # ['a', 'b', 'c', 'd', 'e', 'f', 'g']
a[4:]     #           ['e', 'f', 'g', 'h']
a[-3:]    #           ['f', 'g', 'h']
a[2:5]    #           ['c', 'd', 'e']
a[2:-1]   #           ['c', 'd', 'e', 'f', 'g']
a[-3:-1]  #           ['f', 'g']
```

Slicing deals properly with `start` and `end` indexes that are beyond the boundaries of the list. That makes it easy for your code to establish a maximum length to consider for an input sequence.

```
first_twenty_items = a[:20]
last_twenty_items = a[-20:]
```

In contrast, accessing the same index directly causes an exception.

[Click here to view code image](#)

```
a[20]

>>>
IndexError: list index out of range
```

Note

Beware that indexing a list by a negative variable is one of the few situations in which you can get surprising results from slicing. For example, the expression `somelist[-n:]` will work fine when `n` is greater than one (e.g., `somelist[-3:]`). However, when `n` is zero, the expression `somelist[-0:]` will result in a copy of the original list.

The result of slicing a list is a whole new list. References to the objects from the original list are maintained. Modifying the result of slicing won't affect the original list.

[Click here to view code image](#)

```
b = a[4:]
print('Before: ', b)
b[1] = 99
print('After: ', b)
print('No change:', a)

>>>
Before:  ['e', 'f', 'g', 'h']
After:   ['e', 99, 'g', 'h']
No change: ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

When used in assignments, slices will replace the specified range in the original list. Unlike tuple assignments (like `a, b = c[:2]`), the length of slice assignments don't need to be the same. The values before and after the assigned slice will be preserved. The list will grow or shrink to accommodate the new values.

[Click here to view code image](#)

```
print('Before ', a)
a[2:7] = [99, 22, 14]
print('After ', a)
>>>
Before ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
After  ['a', 'b', 99, 22, 14, 'h']
```

If you leave out both the start and the end indexes when slicing, you'll end up with a copy of the original list.

[Click here to view code image](#)

```
b = a[:]
assert b == a and b is not a
```

If you assign a slice with no start or end indexes, you'll replace its entire contents with a copy of what's referenced (instead of allocating a new list).

[Click here to view code image](#)

```
b = a
print('Before', a)
a[:] = [101, 102, 103]
assert a is b          # Still the same list object
print('After ', a)     # Now has different contents

>>>
Before ['a', 'b', 99, 22, 14, 'h']
After  [101, 102, 103]
```

Things to Remember

- ◆ Avoid being verbose: Don't supply 0 for the start index or the length of the sequence for the end index.
- ◆ Slicing is forgiving of start or end indexes that are out of bounds, making it easy to express slices on the front or back boundaries of a sequence (like `a[:20]` or `a[-20:]`).
- ◆ Assigning to a list slice will replace that range in the original sequence with what's referenced even if their lengths are different.

Item 6: Avoid Using start, end, and stride in a Single Slice

In addition to basic slicing (see [Item 5: “Know How to Slice Sequences”](#)), Python has special syntax for the stride of a slice in the form `somelist[start:end:stride]`. This lets you take every *n*th item when slicing a sequence. For example, the stride makes it easy to group by even and odd indexes in a list.

[Click here to view code image](#)

```
a = ['red', 'orange', 'yellow', 'green', 'blue', 'purple']
odds = a[::2]
evens = a[1::2]
print(odds)
```

```
print(evens)
```

```
>>>
['red', 'yellow', 'blue']
['orange', 'green', 'purple']
```

The problem is that the `stride` syntax often causes unexpected behavior that can introduce bugs. For example, a common Python trick for reversing a byte string is to slice the string with a stride of `-1`.

```
x = b'mongoose'
y = x[::-1]
print(y)
```

```
>>>
b'esoonogm'
```

That works well for byte strings and ASCII characters, but it will break for Unicode characters encoded as UTF-8 byte strings.

[Click here to view code image](#)

```
w = '謝謝'
x = w.encode('utf-8')
y = x[::-1]
z = y.decode('utf-8')
```

```
>>>
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x9d in
position 0: invalid start byte
```

Are negative strides besides `-1` useful? Consider the following examples.

[Click here to view code image](#)

```
a = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
a[::2]    # ['a', 'c', 'e', 'g']
a[::-2]   # ['h', 'f', 'd', 'b']
```

Here, `::2` means select every second item starting at the beginning. Trickier, `::-2` means select every second item starting at the end and moving backwards.

What do you think `2::2` means? What about `-2::-2` vs. `-2:2:-2` vs. `2:2:-2`?

[Click here to view code image](#)

```
a[2::2]    # ['c', 'e', 'g']
a[-2::-2]   # ['g', 'e', 'c', 'a']
a[-2:2:-2]  # ['g', 'e']
a[2:2:-2]   # []
```

The point is that the `stride` part of the slicing syntax can be extremely confusing. Having three numbers within the brackets is hard enough to read because of its density. Then it's not obvious when the `start` and `end` indexes come into effect relative to the `stride` value, especially when `stride` is negative.

To prevent problems, avoid using `stride` along with `start` and `end` indexes. If you must use a `stride`, prefer making it a positive value and omit `start` and `end` indexes. If you must use `stride` with `start` or `end` indexes, consider using one assignment to `stride` and another to slice.

[Click here to view code image](#)

```
b = a[::2] # ['a', 'c', 'e', 'g']
c = b[1:-1] # ['c', 'e']
```

Slicing and then striding will create an extra shallow copy of the data. The first operation should try to reduce the size of the resulting slice by as much as possible. If your program can't afford the time or memory required for two steps, consider using the `itertools` built-in module's `islice` method (see [Item 46: "Use Built-in Algorithms and Data Structures"](#)), which doesn't permit negative values for `start`, `end`, or `stride`.

Things to Remember

- ◆ Specifying `start`, `end`, and `stride` in a slice can be extremely confusing.
- ◆ Prefer using positive `stride` values in slices without `start` or `end` indexes. Avoid negative `stride` values if possible.
- ◆ Avoid using `start`, `end`, and `stride` together in a single slice. If you need all three parameters, consider doing two assignments (one to slice, another to stride) or using `islice` from the `itertools` built-in module.

Item 7: Use List Comprehensions Instead of `map` and `filter`

Python provides compact syntax for deriving one list from another. These expressions are called *list comprehensions*. For example, say you want to compute the square of each number in a list. You can do this by providing the expression for your computation and the input sequence to loop over.

[Click here to view code image](#)

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
squares = [x**2 for x in a]
print(squares)

>>>
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Unless you're applying a single-argument function, list comprehensions are clearer than the `map` built-in function for simple cases. `map` requires creating a `lambda` function for the computation, which is visually noisy.

[Click here to view code image](#)

```
squares = map(lambda x: x ** 2, a)
```

Unlike `map`, list comprehensions let you easily filter items from the input list, removing corresponding outputs from the result. For example, say you only want to compute the squares of the numbers that are divisible by 2. Here, I do this by adding a conditional expression to the list comprehension after the loop:

[Click here to view code image](#)

```
even_squares = [x**2 for x in a if x % 2 == 0]
print(even_squares)
```

```
>>>
[4, 16, 36, 64, 100]
```

The `filter` built-in function can be used along with `map` to achieve the same outcome, but it is much harder to read.

[Click here to view code image](#)

```
alt = map(lambda x: x**2, filter(lambda x: x % 2 == 0, a))
assert even_squares == list(alt)
```

Dictionaries and sets have their own equivalents of list comprehensions. These make it easy to create derivative data structures when writing algorithms.

[Click here to view code image](#)

```
chile_ranks = {'ghost': 1, 'habanero': 2, 'cayenne': 3}
rank_dict = {rank: name for name, rank in chile_ranks.items()}
chile_len_set = {len(name) for name in rank_dict.values()}
print(rank_dict)
print(chile_len_set)
```

```
>>>
{1: 'ghost', 2: 'habanero', 3: 'cayenne'}
{8, 5, 7}
```

Things to Remember

- ◆ List comprehensions are clearer than the `map` and `filter` built-in functions because they don't require extra `lambda` expressions.
- ◆ List comprehensions allow you to easily skip items from the input list, a behavior `map` doesn't support without help from `filter`.
- ◆ Dictionaries and sets also support comprehension expressions.

Item 8: Avoid More Than Two Expressions in List Comprehensions

Beyond basic usage (see [Item 7: “Use List Comprehensions Instead of `map` and `filter`”](#)), list comprehensions also support multiple levels of looping. For example, say you want to simplify a matrix (a list containing other lists) into one flat list of all cells. Here, I do this with a list comprehension by including two `for` expressions. These expressions run in the order provided from left to right.

[Click here to view code image](#)

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flat = [x for row in matrix for x in row]
print(flat)
```

```
>>>
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

The example above is simple, readable, and a reasonable usage of multiple loops. Another reasonable usage of multiple loops is replicating the two-level deep layout of the input list.

For example, say you want to square the value in each cell of a two-dimensional matrix. This expression is noisier because of the extra `[]` characters, but it's still easy to read.

[Click here to view code image](#)

```
squared = [[x**2 for x in row] for row in matrix]
print(squared)

>>>
[[1, 4, 9], [16, 25, 36], [49, 64, 81]]
```

If this expression included another loop, the list comprehension would get so long that you'd have to split it over multiple lines.

[Click here to view code image](#)

```
my_lists = [
    [[1, 2, 3], [4, 5, 6]],
    # ...
]
flat = [x for sublist1 in my_lists
        for sublist2 in sublist1
        for x in sublist2]
```

At this point, the multiline comprehension isn't much shorter than the alternative. Here, I produce the same result using normal loop statements. The indentation of this version makes the looping clearer than the list comprehension.

```
flat = []
for sublist1 in my_lists:
    for sublist2 in sublist1:
        flat.extend(sublist2)
```

List comprehensions also support multiple `if` conditions. Multiple conditions at the same loop level are an implicit `and` expression. For example, say you want to filter a list of numbers to only even values greater than four. These two list comprehensions are equivalent.

[Click here to view code image](#)

```
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
b = [x for x in a if x > 4 if x % 2 == 0]
c = [x for x in a if x > 4 and x % 2 == 0]
```

Conditions can be specified at each level of looping after the `for` expression. For example, say you want to filter a matrix so the only cells remaining are those divisible by 3 in rows that sum to 10 or higher. Expressing this with list comprehensions is short, but extremely difficult to read.

[Click here to view code image](#)

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
filtered = [[x for x in row if x % 3 == 0]
            for row in matrix if sum(row) >= 10]
print(filtered)

>>>
[[6], [9]]
```

Though this example is a bit convoluted, in practice you'll see situations arise where such expressions seem like a good fit. I strongly encourage you to avoid using list

comprehensions that look like this. The resulting code is very difficult for others to comprehend. What you save in the number of lines doesn't outweigh the difficulties it could cause later.

The rule of thumb is to avoid using more than two expressions in a list comprehension. This could be two conditions, two loops, or one condition and one loop. As soon as it gets more complicated than that, you should use normal `if` and `for` statements and write a helper function (see [Item 16](#): “[Consider Generators Instead of Returning Lists](#)”).

Things to Remember

- ◆ List comprehensions support multiple levels of loops and multiple conditions per loop level.
- ◆ List comprehensions with more than two expressions are very difficult to read and should be avoided.

Item 9: Consider Generator Expressions for Large Comprehensions

The problem with list comprehensions (see [Item 7](#): “[Use List Comprehensions Instead of map and filter](#)”) is that they may create a whole new list containing one item for each value in the input sequence. This is fine for small inputs, but for large inputs this could consume significant amounts of memory and cause your program to crash.

For example, say you want to read a file and return the number of characters on each line. Doing this with a list comprehension would require holding the length of every line of the file in memory. If the file is absolutely enormous or perhaps a never-ending network socket, list comprehensions are problematic. Here, I use a list comprehension in a way that can only handle small input values.

[Click here to view code image](#)

```
value = [len(x) for x in open('/tmp/my_file.txt')]
print(value)
```

```
>>>
[100, 57, 15, 1, 12, 75, 5, 86, 89, 11]
```

To solve this, Python provides *generator expressions*, a generalization of list comprehensions and generators. Generator expressions don't materialize the whole output sequence when they're run. Instead, generator expressions evaluate to an iterator that yields one item at a time from the expression.

A generator expression is created by putting list-comprehension-like syntax between `( )` characters. Here, I use a generator expression that is equivalent to the code above. However, the generator expression immediately evaluates to an iterator and doesn't make any forward progress.

[Click here to view code image](#)

```
it = (len(x) for x in open('/tmp/my_file.txt'))
print(it)
```

```
>>>
<generator object <genexpr> at 0x101b81480>
```

The returned iterator can be advanced one step at a time to produce the next output from the generator expression as needed (using the `next` built-in function). Your code can consume as much of the generator expression as you want without risking a blowup in memory usage.

```
print(next(it))
print(next(it))
```

```
>>>
100
57
```

Another powerful outcome of generator expressions is that they can be composed together. Here, I take the iterator returned by the generator expression above and use it as the input for another generator expression.

[Click here to view code image](#)

```
roots = ((x, x**0.5) for x in it)
```

Each time I advance this iterator, it will also advance the interior iterator, creating a domino effect of looping, evaluating conditional expressions, and passing around inputs and outputs.

```
print(next(roots))
```

```
>>>
(15, 3.872983346207417)
```

Chaining generators like this executes very quickly in Python. When you're looking for a way to compose functionality that's operating on a large stream of input, generator expressions are the best tool for the job. The only gotcha is that the iterators returned by generator expressions are stateful, so you must be careful not to use them more than once (see [Item 17: "Be Defensive When Iterating Over Arguments"](#)).

Things to Remember

- ◆ List comprehensions can cause problems for large inputs by using too much memory.
- ◆ Generator expressions avoid memory issues by producing outputs one at a time as an iterator.
- ◆ Generator expressions can be composed by passing the iterator from one generator expression into the `for` subexpression of another.
- ◆ Generator expressions execute very quickly when chained together.

Item 10: Prefer `enumerate` Over `range`

The `range` built-in function is useful for loops that iterate over a set of integers.

```
random_bits = 0
for i in range(64):
```

```
if randint(0, 1):
    random_bits |= 1 << i
```

When you have a data structure to iterate over, like a list of strings, you can loop directly over the sequence.

[Click here to view code image](#)

```
flavor_list = ['vanilla', 'chocolate', 'pecan', 'strawberry']
for flavor in flavor_list:
    print('%s is delicious' % flavor)
```

Often, you'll want to iterate over a list and also know the index of the current item in the list. For example, say you want to print the ranking of your favorite ice cream flavors. One way to do it is using `range`.

[Click here to view code image](#)

```
for i in range(len(flavor_list)):
    flavor = flavor_list[i]
    print('%d: %s' % (i + 1, flavor))
```

This looks clumsy compared with the other examples of iterating over `flavor_list` or `range`. You have to get the length of the list. You have to index into the array. It's harder to read.

Python provides the `enumerate` built-in function for addressing this situation. `enumerate` wraps any iterator with a lazy generator. This generator yields pairs of the loop index and the next value from the iterator. The resulting code is much clearer.

[Click here to view code image](#)

```
for i, flavor in enumerate(flavor_list):
    print('%d: %s' % (i + 1, flavor))
```

```
>>>
1: vanilla
2: chocolate
3: pecan
4: strawberry
```

You can make this even shorter by specifying the number from which `enumerate` should begin counting (1 in this case).

[Click here to view code image](#)

```
for i, flavor in enumerate(flavor_list, 1):
    print('%d: %s' % (i, flavor))
```

Things to Remember

- ◆ `enumerate` provides concise syntax for looping over an iterator and getting the index of each item from the iterator as you go.
- ◆ Prefer `enumerate` instead of looping over a `range` and indexing into a sequence.
- ◆ You can supply a second parameter to `enumerate` to specify the number from which to begin counting (zero is the default).

Item 11: Use `zip` to Process Iterators in Parallel

Often in Python you find yourself with many lists of related objects. List comprehensions make it easy to take a source list and get a derived list by applying an expression (see [Item 7: “Use List Comprehensions Instead of `map` and `filter`”](#)).

[Click here to view code image](#)

```
names = ['Cecilia', 'Lise', 'Marie']
letters = [len(n) for n in names]
```

The items in the derived list are related to the items in the source list by their indexes. To iterate over both lists in parallel, you can iterate over the length of the `names` source list.

[Click here to view code image](#)

```
longest_name = None
max_letters = 0

for i in range(len(names)):
    count = letters[i]
    if count > max_letters:
        longest_name = names[i]
        max_letters = count

print(longest_name)

>>>
Cecilia
```

The problem is that this whole loop statement is visually noisy. The indexes into `names` and `letters` make the code hard to read. Indexing into the arrays by the loop index `i` happens twice. Using `enumerate` (see [Item 10: “Prefer `enumerate` Over `range`”](#)) improves this slightly, but it’s still not ideal.

[Click here to view code image](#)

```
for i, name in enumerate(names):
    count = letters[i]
    if count > max_letters:
        longest_name = name
        max_letters = count
```

To make this code clearer, Python provides the `zip` built-in function. In Python 3, `zip` wraps two or more iterators with a lazy generator. The `zip` generator yields tuples containing the next value from each iterator. The resulting code is much cleaner than indexing into multiple lists.

[Click here to view code image](#)

```
for name, count in zip(names, letters):
    if count > max_letters:
        longest_name = name
        max_letters = count
```

There are two problems with the `zip` built-in.

The first issue is that in Python 2 `zip` is not a generator; it will fully exhaust the supplied iterators and return a list of all the tuples it creates. This could potentially use a lot of memory and cause your program to crash. If you want to `zip` very large iterators in

Python 2, you should use `izip` from the `itertools` built-in module (see [Item 46: “Use Built-in Algorithms and Data Structures”](#)).

The second issue is that `zip` behaves strangely if the input iterators are of different lengths. For example, say you add another name to the list above but forget to update the letter counts. Running `zip` on the two input lists will have an unexpected result.

[Click here to view code image](#)

```
names.append('Rosalind')
for name, count in zip(names, letters):
    print(name)
```

```
>>>
Cecilia
Lise
Marie
```

The new item for 'Rosalind' isn't there. This is just how `zip` works. It keeps yielding tuples until a wrapped iterator is exhausted. This approach works fine when you know that the iterators are of the same length, which is often the case for derived lists created by list comprehensions. In many other cases, the truncating behavior of `zip` is surprising and bad. If you aren't confident that the lengths of the lists you want to `zip` are equal, consider using the `zip_longest` function from the `itertools` built-in module instead (also called `izip_longest` in Python 2).

Things to Remember

- ◆ The `zip` built-in function can be used to iterate over multiple iterators in parallel.
- ◆ In Python 3, `zip` is a lazy generator that produces tuples. In Python 2, `zip` returns the full result as a list of tuples.
- ◆ `zip` truncates its output silently if you supply it with iterators of different lengths.
- ◆ The `zip_longest` function from the `itertools` built-in module lets you iterate over multiple iterators in parallel regardless of their lengths (see [Item 46: “Use Built-in Algorithms and Data Structures”](#)).

Item 12: Avoid `else` Blocks After `for` and `while` Loops

Python loops have an extra feature that is not available in most other programming languages: you can put an `else` block immediately after a loop's repeated interior block.

```
for i in range(3):
    print('Loop %d' % i)
else:
    print('Else block!')
```

```
>>>
Loop 0
Loop 1
Loop 2
Else block!
```

Surprisingly, the `else` block runs immediately after the loop finishes. Why is the clause called “`else`”? Why not “`and`”? In an `if/else` statement, `else` means, “Do this if the block before this doesn’t happen.” In a `try/except` statement, `except` has the same definition: “Do this if trying the block before this failed.”

Similarly, `else` from `try/except/else` follows this pattern (see [Item 13: “Take Advantage of Each Block in try/except/else/finally”](#)) because it means, “Do this if the block before did not fail.” `try/finally` is also intuitive because it means, “Always do what is final after trying the block before.”

Given all of the uses of `else`, `except`, and `finally` in Python, a new programmer might assume that the `else` part of `for/else` means, “Do this if the loop wasn’t completed.” In reality, it does exactly the opposite. Using a `break` statement in a loop will actually skip the `else` block.

```
for i in range(3):
    print('Loop %d' % i)
    if i == 1:
        break
else:
    print('Else block!')
```

```
>>>
Loop 0
Loop 1
```

Another surprise is that the `else` block will run immediately if you loop over an empty sequence.

[Click here to view code image](#)

```
for x in []:
    print('Never runs')
else:
    print('For Else block!')
```

```
>>>
For Else block!
```

The `else` block also runs when `while` loops are initially false.

[Click here to view code image](#)

```
while False:
    print('Never runs')
else:
    print('While Else block!')
```

```
>>>
While Else block!
```

The rationale for these behaviors is that `else` blocks after loops are useful when you’re using loops to search for something. For example, say you want to determine whether two numbers are coprime (their only common divisor is 1). Here, I iterate through every possible common divisor and test the numbers. After every option has been tried, the loop ends. The `else` block runs when the numbers are coprime because the loop doesn’t encounter a `break`.

[Click here to view code image](#)

```
a = 4
b = 9
for i in range(2, min(a, b) + 1):
    print('Testing', i)
    if a % i == 0 and b % i == 0:
        print('Not coprime')
        break
else:
    print('Coprime')

>>>
Testing 2
Testing 3
Testing 4
Coprime
```

In practice, you wouldn't write the code this way. Instead, you'd write a helper function to do the calculation. Such a helper function is written in two common styles.

The first approach is to return early when you find the condition you're looking for. You return the default outcome if you fall through the loop.

[Click here to view code image](#)

```
def coprime(a, b):
    for i in range(2, min(a, b) + 1):
        if a % i == 0 and b % i == 0:
            return False
    return True
```

The second way is to have a result variable that indicates whether you've found what you're looking for in the loop. You **break** out of the loop as soon as you find something.

[Click here to view code image](#)

```
def coprime2(a, b):
    is_coprime = True
    for i in range(2, min(a, b) + 1):
        if a % i == 0 and b % i == 0:
            is_coprime = False
            break
    return is_coprime
```

Both of these approaches are so much clearer to readers of unfamiliar code. The expressivity you gain from the **else** block doesn't outweigh the burden you put on people (including yourself) who want to understand your code in the future. Simple constructs like loops should be self-evident in Python. You should avoid using **else** blocks after loops entirely.

Things to Remember

- ◆ Python has special syntax that allows **else** blocks to immediately follow **for** and **while** loop interior blocks.
- ◆ The **else** block after a loop only runs if the loop body did not encounter a **break** statement.

- ◆ Avoid using `else` blocks after loops because their behavior isn't intuitive and can be confusing.

Item 13: Take Advantage of Each Block in `try/except/else/finally`

There are four distinct times that you may want to take action during exception handling in Python. These are captured in the functionality of `try`, `except`, `else`, and `finally` blocks. Each block serves a unique purpose in the compound statement, and their various combinations are useful (see [Item 51: “Define a Root Exception to Insulate Callers from APIs”](#) for another example).

Finally Blocks

Use `try/finally` when you want exceptions to propagate up, but you also want to run cleanup code even when exceptions occur. One common usage of `try/finally` is for reliably closing file handles (see [Item 43: “Consider `contextlib` and `with` Statements for Reusable `try/finally` Behavior”](#) for another approach).

[Click here to view code image](#)

```
handle = open('/tmp/random_data.txt') # May raise IOError
try:
    data = handle.read() # May raise UnicodeDecodeError
finally:
    handle.close() # Always runs after try:
```

Any exception raised by the `read` method will always propagate up to the calling code, yet the `close` method of `handle` is also guaranteed to run in the `finally` block. You must call `open` before the `try` block because exceptions that occur when opening the file (like `IOError` if the file does not exist) should skip the `finally` block.

Else Blocks

Use `try/except/else` to make it clear which exceptions will be handled by your code and which exceptions will propagate up. When the `try` block doesn't raise an exception, the `else` block will run. The `else` block helps you minimize the amount of code in the `try` block and improves readability. For example, say you want to load JSON dictionary data from a string and return the value of a key it contains.

[Click here to view code image](#)

```
def load_json_key(data, key):
    try:
        result_dict = json.loads(data) # May raise ValueError
    except ValueError as e:
        raise KeyError from e
    else:
        return result_dict[key] # May raise KeyError
```

If the data isn't valid JSON, then decoding with `json.loads` will raise a `ValueError`. The exception is caught by the `except` block and handled. If decoding is

successful, then the key lookup will occur in the `else` block. If the key lookup raises any exceptions, they will propagate up to the caller because they are outside the `try` block. The `else` clause ensures that what follows the `try/except` is visually distinguished from the `except` block. This makes the exception propagation behavior clear.

Everything Together

Use `try/except/else/finally` when you want to do it all in one compound statement. For example, say you want to read a description of work to do from a file, process it, and then update the file in place. Here, the `try` block is used to read the file and process it. The `except` block is used to handle exceptions from the `try` block that are expected. The `else` block is used to update the file in place and to allow related exceptions to propagate up. The `finally` block cleans up the file handle.

[Click here to view code image](#)

```
UNDEFINED = object()

def divide_json(path):
    handle = open(path, 'r+')    # May raise IOError
    try:
        data = handle.read()    # May raise UnicodeDecodeError
        op = json.loads(data)    # May raise ValueError
        value = (
            op['numerator'] /
            op['denominator'])  # May raise ZeroDivisionError
    except ZeroDivisionError as e:
        return UNDEFINED
    else:
        op['result'] = value
        result = json.dumps(op)
        handle.seek(0)
        handle.write(result)    # May raise IOError
        return value
    finally:
        handle.close()          # Always runs
```

This layout is especially useful because all of the blocks work together in intuitive ways. For example, if an exception gets raised in the `else` block while rewriting the result data, the `finally` block will still run and close the file handle.

Things to Remember

- ◆ The `try/finally` compound statement lets you run cleanup code regardless of whether exceptions were raised in the `try` block.
- ◆ The `else` block helps you minimize the amount of code in `try` blocks and visually distinguish the success case from the `try/except` blocks.
- ◆ An `else` block can be used to perform additional actions after a successful `try` block but before common cleanup in a `finally` block.